**Límite de entrega**: domingo, 26 de octubre de 2025, 12:30
**Ficheros requeridos**: transporte.py, prueba.py (Descargar)
**Tipo de trabajo**: Individual

## Clase `TarjetaTransporte`

Desarrolle en el módulo `transporte` una clase en Python de nombre `TarjetaTransporte` que modele una tarjeta recargable de transporte público, con las siguientes características y requerimientos:

---

## Clase `TarjetaTransporte`

### Consideraciones generales

- Añada los atributos de instancia y de clase que considere necesarios.
- Los atributos deben estar encapsulados correctamente.
- No se requieren más validaciones que las indicadas.

---

## Inicializador (constructor)

- Recibe el nombre del usuario titular de la tarjeta (string).

---

## Propiedades de instancia

- `numero_tarjeta` (solo lectura): string, código único con formato `"TT-nnnnnnn"`, donde `nnnnnnn` son números consecutivos con siete cifras empezando por 1000001 (sigue 1000002, 1000003, ...).
- `titular` (solo lectura): nombre del usuario.
- `saldo` (solo lectura): float, saldo disponible (inicialmente `0.0`).
- `viajes_realizados` (solo lectura): entero, número de viajes realizados con la tarjeta (inicialmente `0`).

---

## Métodos de instancia

- `recargar(cantidad)`: Añade la cantidad al saldo, salvo que sea negativa, en cuyo caso lanza una excepción `ValueError`.
- `viajar(costo_viaje)`: Descuenta el costo del viaje del saldo. Además, incrementa en 1 los viajes realizados por el titular con dicha tarjeta. Si no hay saldo suficiente o el costo es negativo, solo lanza una excepción `ValueError`.

---

## Métodos de clase

- `total_tarjetas_emitidas()`: Devuelve cuántas tarjetas han sido creadas. en total
- `viajes_titular(nombre_titular)`: Devuelve el número total de viajes realizados con cualquier tarjeta por el titular. Si no ha tenido tarjetas 0.

---

## Representación informal

- La representación informal de la tarjeta será una str con el siguiente formato:
  `<numero_tarjeta> - Titular: <titular>, Saldo: <saldo>, Viajes: <numero_viajes>`
  Donde `<numero_tarjeta>` es el número de tarjeta, `<titular>` es el titular de la tarjeta y `<numero_viajes>` es el número de viajes realizados con la tarjeta.
  Para una tarjeta se podría tener la siguiente representación informal:
  `TT-1000031 - Titular: Ana López, Saldo: 1.5, Viajes: 2`

---

?

## Ejemplo de uso

```
t1 = TarjetaTransporte("Ana López")
t1.recargar(20.0)
t1.viajar(2.5)
t1.viajar(2.5)
print(t1)
# TP-1000001 - Titular: Ana López, Saldo: 15.0, Viajes: 2
t2 = TarjetaTransporte("Carlos Ruiz")
t2.recargar(10.0)
print(t2)
# TP-1000002 - Titular: Carlos Ruiz, Saldo: 10.0, Viajes: 0
```

## 🧪 Pruebas

Se dispone de un fichero `prueba.py` modificable para probar su clase al ejecutarlo.

**Notas**: Se recomienda desplegar la pestaña Descripción justo antes de pulsar Evaluar, y desplegar también la pestaña Comentarios aun cuando se muestre algún mensaje en la de Compilación.

©2012-2015 - Laurent Pointal Mémento v2.0.6
License Creative Commons Attribution 4

# Python 3 Cheat Sheet

*Latest version on :*
*https://perso.limsi.fr/pointal/python:memento*

## Base Types

*integer, float, boolean, string, bytes*

```
int 783  0 -192  0b010 0o642 0xF3
float 9.23 0.0  -1.7e-6
bool True False
str "One\nTwo"
    'I\'m'
bytes b"toto\xfe\775"
```

*zero* *binary* *octal* *hexa*

*escaped new line* *escaped '* *escaped tab* *hexadecimal* *octal* *$\times 10^{-6}$*

*Multiline string:* `"""X\tY\tZ 1\t2\t3"""`

*immutables*

## Container Types

- **ordered sequences**, fast index access, repeatable values

```
list [1,5,9]   ["x",11,8.9]   ["mot"]   []
tuple (1,5,9)  11,"y",7.4     ("mot",)  ()
str bytes
```

*Non modifiable values (immutables)* *expression with only comas → `tuple`* *(ordered sequences of chars / bytes)*

- **key containers**, no *a priori* order, fast key access, each key is unique

**dictionary** `dict {"key":"value"}` `dict(a=3,b=4,k="v")` `{}`
*(key/value associations)* `{1:"one",3:"three",2:"two",3.14:"π"}`

**collection** `set {"key1","key2"}` `{1,9,3,0}` `set()`

*keys=hashable values (base types, immutables…)* `frozenset` *immutable set* *empty*

## Identifiers

*for variables, functions, modules, classes… names*

`a…zA…Z_` followed by `a…zA…Z_0…9`
- diacritics allowed but should be avoided
- language keywords forbidden
- lower/UPPER case discrimination

☺ `a toto x7 y_max BigOne`
☹ `~~By and for~~`

## Variables assignment

`=`

assignment ⇔ **binding** of a *name* with a *value*
1) *evaluation of right side expression value*
2) *assignment in order with left side names*

```
x=1.2+8+sin(y)
a=b=c=0      assignment to same value
y,z,r=9.2,-7.6,0   multiple assignments
a,b=b,a      values swap
a,*b=seq     unpacking of sequence in
*a,b=seq     item and list
x+=3         increment ⇔ x=x+3
x-=2         decrement ⇔ x=x-2
x=None       « undefined » constant value
del x        remove name x
```

and `*=` `/=` `%=` …

## Conversions

`type(expression)`

```
int("15") → 15
int("3f",16) → 63          can specify integer number base in 2nd parameter
int(15.56) → 15            truncate decimal part
float("-11.24e8") → -1124000000.0
round(15.56,1)→15.6        rounding to 1 decimal (0 decimal → integer number)
bool(x)  False for null x, empty container x , None or False x ; True for other x
str(x)→ "…"   representation string of x for display (cf. formatting on the back)
chr(64)→'@'  ord('@')→64   code ↔ char
repr(x)→ "…"  literal representation string of x
bytes([72,9,64]) → b'H\t@'
list("abc") → ['a','b','c']
dict([(3,"three"),(1,"one")]) → {1:'one',3:'three'}
set(["one","two"]) → {'one','two'}
separator str and sequence of str → assembled str
  ':'.join(['toto','12','pswd']) → 'toto:12:pswd'
str splitted on whitespaces → list of str
  "words with  spaces".split() → ['words','with','spaces']
str splitted on separator str → list of str
  "1,4,8,2".split(",") → ['1','4','8','2']
sequence of one type → list of another type (via list comprehension)
  [int(x) for x in ('1','29','-3')] → [1,29,-3]
```

## Sequence Containers Indexing

*for lists, tuples, strings, bytes…*

| | | | | | |
|---|---|---|---|---|---|
| *negative index* | −5 | −4 | −3 | −2 | −1 |
| *positive index* | 0 | 1 | 2 | 3 | 4 |
| `lst=[10,` | `20,` | `30,` | `40,` | `50]` | |
| *positive slice* | 0 | 1 | 2 | 3 | 4 | 5 |
| *negative slice* | −5 | −4 | −3 | −2 | −1 | |

**Items count**
`len(lst)→5`

**index from 0**
(here from 0 to 4)

Individual access to **items** via `lst[index]`
`lst[0]→10` ⇒ *first one* `lst[1]→20`
`lst[-1]→50` ⇒ *last one* `lst[-2]→40`

*On mutable sequences (`list`), remove with `del lst[3]` and modify with assignment `lst[4]=25`*

Access to **sub-sequences** via `lst[start slice:end slice:step]`

```
lst[:-1]→[10,20,30,40]   lst[::-1]→[50,40,30,20,10]   lst[1:3]→[20,30]   lst[:3]→[10,20,30]
lst[1:-1]→[20,30,40]     lst[::-2]→[50,30,10]         lst[-3:-1]→[30,40] lst[3:]→[40,50]
lst[::2]→[10,30,50]      lst[:]→[10,20,30,40,50] shallow copy of sequence
```

*Missing slice indication → from start / up to end.*
*On mutable sequences (`list`), remove with `del lst[3:5]` and modify with assignment `lst[1:4]=[15,25]`*

## Boolean Logic

Comparisons : `< > <= >= == !=` (*boolean results*) ≤ ≥ = ≠

`a and b` logical and *both simultaneously*

`a or b` logical or *one or other or both*

pitfall : `and` *and* `or` *return value of* `a` *or of* `b` *(under shortcut evaluation).*
⇒ *ensure that* `a` *and* `b` *are booleans.*

`not a` logical not

`True` `False` True and False constants

## Statements Blocks

*parent statement :*
*statement block 1…*
⋮
*parent statement :*
*statement block2…*
⋮
*next statement after block 1*

indentation

*configure editor to insert 4 spaces in place of an indentation tab.*

## Modules/Names Imports

*module `truc`⇔file `truc.py`*

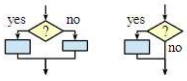
```
from monmod import nom1,nom2 as fct
  →direct access to names, renaming with as
import monmod →access via monmod.nom1 …
```

modules and packages searched in *python path* (cf `sys.path`)

## Conditional Statement

*statement block executed only if a condition is true*

```
if logical condition :
    statements block
```




Can go with several *elif*, *elif*… and only one final *else*. Only the block of first true condition is executed.

```
if age<=18:
  state="Kid"
elif age>65:
  state="Retired"
else:
  state="Active"
```

*with a var `x`:*
`if bool(x)==True:` ⇔ `if x:`
`if bool(x)==False:` ⇔ `if not x:`

## Maths

*floating numbers… approximated values* *angles in radians*

Operators: `+ - * / // % **`
Priority `(…)` × ÷ ↑ ↑ $a^b$ integer ÷ ÷ remainder

`@` → matrix × *python3.5+`numpy`*

```
(1+5.3)*2→12.6
abs(-3.2)→3.2
round(3.57,1)→3.6
pow(4,3)→64.0
```

*usual order of operations*

```
from math import sin,pi…
sin(pi/4)→0.707…
cos(2*pi/3)→-0.4999…
sqrt(81)→9.0  √
log(e**2)→2.0
ceil(12.5)→13
floor(12.5)→12
```

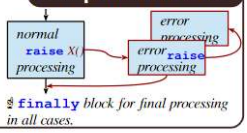
modules `math`, `statistics`, `random`, `decimal`, `fractions`, `numpy`, *etc. (cf. doc)*

## Exceptions on Errors

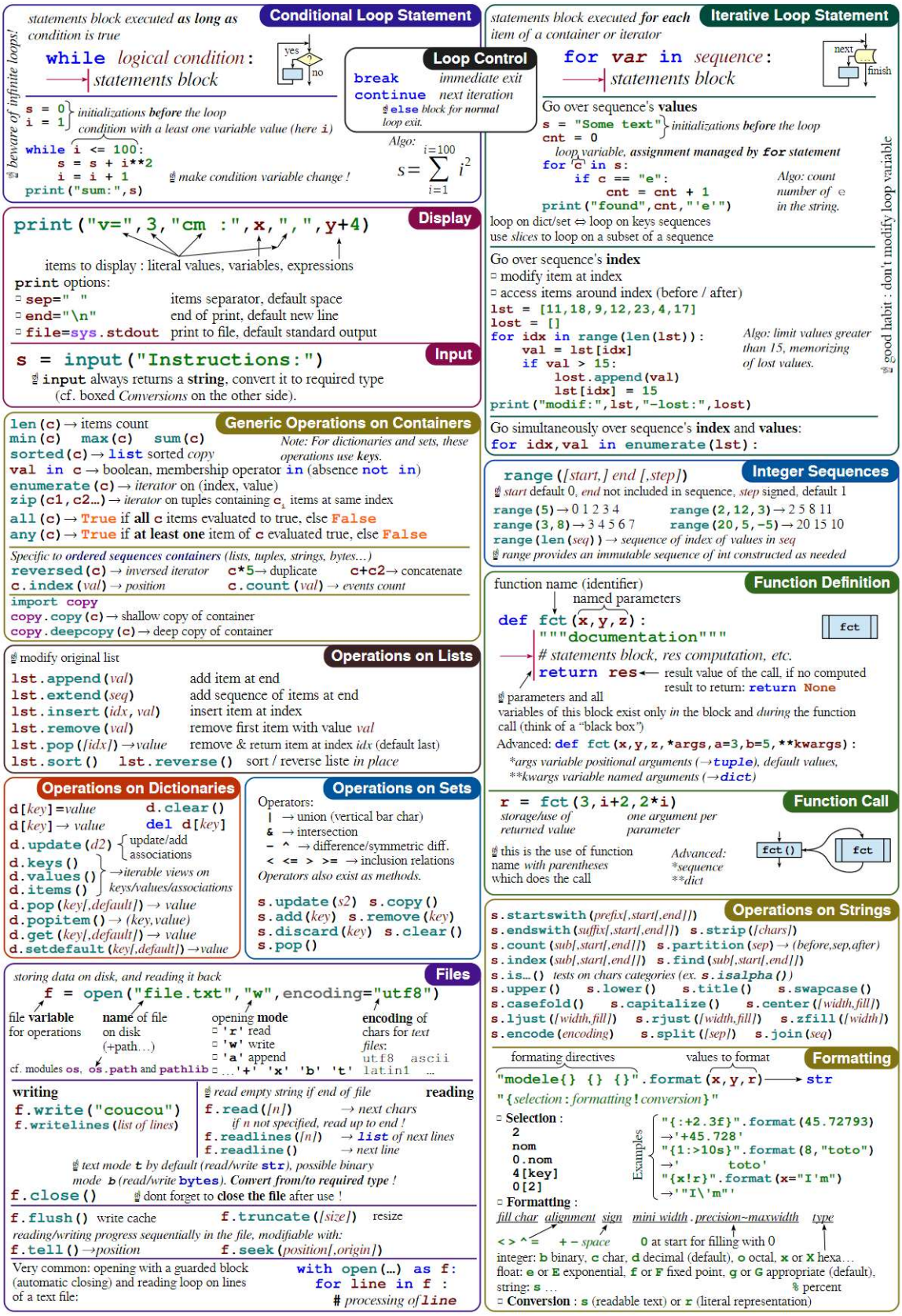
Signaling an error:
`raise` *ExcClass(…)*

Errors processing:

```
try:
    normal procesising block
except Exception as e:
    error processing block
```




`finally` *block for final processing in all cases.*




## Ficheros requeridos

### transporte.py

```
# ESCRIBA AQUÍ LA CLASE
class TarjetaTransporte:
    # descomente la siguiente línea si quiere usarla
    # __viajes_titular: dict[str, int] = {}  # titular: número de viajes

```

### prueba.py

```python
# PROBANDO la clase TarjetaTransporte:
from transporte import TarjetaTransporte

# Crear varias tarjetas para distintos titulares
t1 = TarjetaTransporte("Ana López")
t2 = TarjetaTransporte("Carlos Ruiz")
t3 = TarjetaTransporte("Ana López")

# Mostrar información básica de cada tarjeta
print(t1)
print(t2)
print(t3)

# Recargar una tarjeta y realizar viajes
t1.recargar(20)
t1.viajar(2.5)
t1.viajar(3.0)
print(f"Después de viajar dos veces: {t1}")

# Intentar hacer un viaje sin saldo suficiente
try:
    t2.viajar(5.0)
except ValueError as e:
    print(f"Error al viajar sin saldo: {e}")

# Recargar y viajar correctamente
t2.recargar(10)
t2.viajar(2.5)
print(f"Después de recarga y viaje: {t2}")

# Consultar cuántas tarjetas se han emitido
print(f"Total de tarjetas emitidas: {TarjetaTransporte.total_tarjetas_emitidas()}")
t3.recargar(2.5)
t3.viajar(2.5)

# Consultar viajes total por titular
viajes_ana = TarjetaTransporte.viajes_titular("Ana López")
viajes_carlos = TarjetaTransporte.viajes_titular("Carlos Ruiz")
viajes_ines = TarjetaTransporte.viajes_titular("Inés Morales")  # titular inexistente

print(f"Total viajes Ana López: {saldo_ana}")
print(f"Total viajes Carlos Ruiz: {saldo_carlos}")
print(f"Total viajes Inés Morales: {saldo_ines}")
```

VPL